

Ticket_fail

좌석 초과 예약을 일부러 만들고, 네 가지 방법으로 고친 기록

동시성 제어 포트폴리오 프로젝트 · Spring Boot · Postgres · Redis

github.com/yunok96/Ticket_fail · 2026년 9월

이 프로젝트를 왜 만들었나

코드를 잘 짜는 걸 보여주는 프로젝트가 아니라, 판단을 설명할 수 있게 만드는 프로젝트

목표

영국 백엔드 채용을 겨냥한 포트폴리오. 티켓팅 주제 자체는 흔하므로, 주제가 아니라 락 전략의 비교가 내용물이 된다.

방식

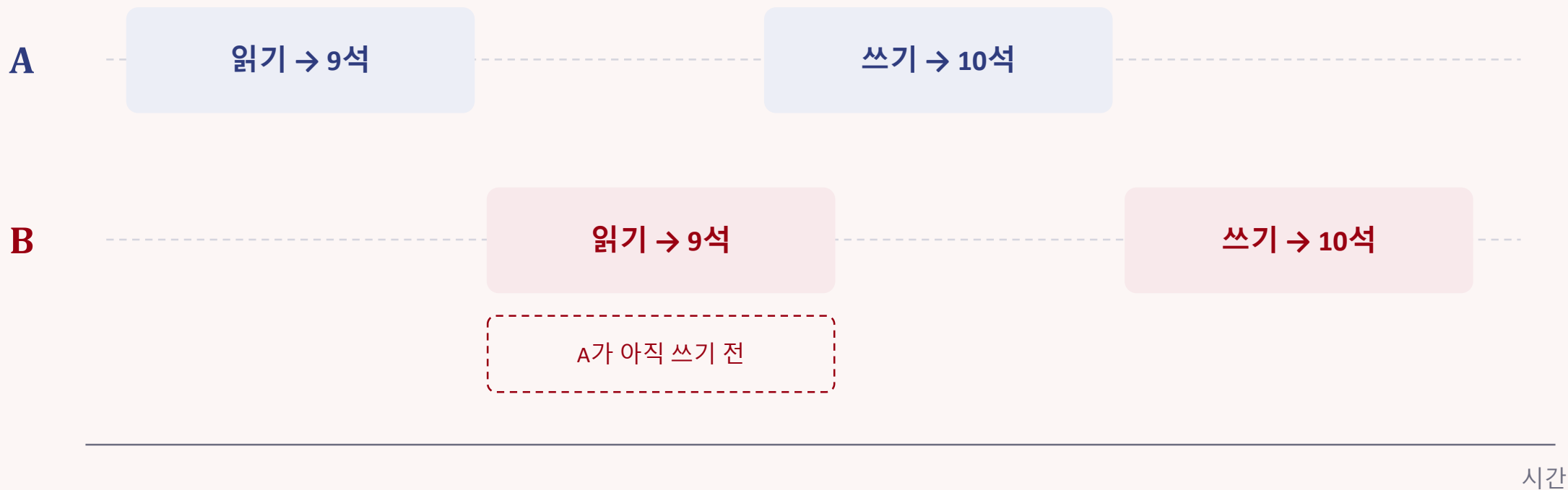
먼저 일부러 고장난 버전을 만들어 초과 예약을 눈으로 확인하고, 그다음 락을 하나씩 붙여가며 무엇이 달라지는지 본다.

산출물

락 전략 다섯 개가 각각 독립적으로 실행되는 레포와, 각 전략이 같은 부하에서 어떤 숫자를 내는지 정리한 비교 표.

문제: 읽고 · 판단하고 · 쓰는 사이의 틈

좌석 예약은 한 동작처럼 보이지만 실제로는 세 동작이다



두 요청 모두 9를 읽었으므로 둘 다 10을 쓴다. 예약은 2건이 기록되지만 좌석은 1석만 늘어난다.

읽는 시점과 쓰는 시점 사이에 남이 끼어들 틈이 있다 — check-then-act 문제.

1

락 없음 — 고장난 상태를 먼저 만든다

이 단계의 테스트는 통과가 아니라 실패를 증명한다

100

동시 요청

100

예약 성공

1

실제 좌석 수

매진 검사는 한 번도 걸리지 않았다

좌석 카운터가 한계에 가까워진 적이 없기 때문이다. 100명이 각각 "아직 9석 남았네"를 보고 통과했다.

테스트는 카운터와 성공 건수가 서로 어긋난다는 것을 단언한다 — 버그를 설명하는 대신 고정시켜 둔 것이다.

@Transactional은 왜 이걸 못 막나

이 프로젝트에서 가장 자주 오해되는 지점

트랜잭션이 하는 일

아직 커밋되지 않은 작업을 남에게 감춘다.
중간 상태가 보이지 않게 해준다.

트랜잭션이 하지 않는 일

동시에 들어온 요청을 줄 세우지 않는다.
각자 읽은 값은 읽는 순간엔 정당하게 최신이다.

그리고 JPA가 쓰는 SQL이 문제를 낳는다

`SET reserved_seats = <자바에서 계산한 값>`

`SET reserved_seats = reserved_seats + 1`

계산해서 덮어쓰기 때문에, 나중에 쓴 값이 앞의 값을 지운다.

← 더티 체킹이 만드는 UPDATE

← 원자적 증가였다면

2

비관적 락 — 읽는 순간에 잠근다

충돌은 반드시 난다고 보고 미리 막는 방식

```
SELECT ... FOR UPDATE          @Lock(LockModeType.PESSIMISTIC_WRITE)
```

어떻게

행을 쓸 때가 아니라 읽을 때 잠그고,
트랜잭션이 커밋될 때까지 유지한다.
두 번째 요청은 첫 번째가 쓰기를
끝낼 때까지 카운터를 읽지 못한다.

결과

요청 100건 중 10건 성공,
90건 매진.
좌석 카운터 정확히 10.

대가

같은 공연에 대한 요청이
한 줄로 세워진다.
처리량이 떨어진다.

짚어줄 점 — 락이 없던 버전도 사실 락을 걸고 있었다. 모든 UPDATE는 자기가 건드리는 행을 잠그기 때문이다.
다만 잘못된 값이 이미 계산된 뒤에 걸렸을 뿐이다. 이 단계는 락을 추가한 게 아니라, 있던 락을 앞으로 당긴 것이다.

3

낙관적 락 — 아무것도 잠그지 않는다

충돌은 드물다고 보고 일단 통과시킨 뒤, 나중에 확인한다

```
@Version → UPDATE ... WHERE id = ? AND version = ?
```

어떻게

버전 칼럼을 두고, 갱신할 때
"내가 읽었을 때와 같은가"를 확인한다
.
경쟁에서 진 요청은 0건을 갱신하고
예외를 받는다.

결과

100건 중 10건 성공,
59건 매진, 31건 충돌.
좌석 카운터는 여전히 10.

한계

좌석이 남아 있는데도 거절당한
요청이 생긴다. 정합성은 맞지만
서비스로는 손해다.

재시도 래퍼가 필요한 이유 — 버전 충돌은 커밋 시점에, 즉 프록시 안쪽에서 원래 메서드가 끝난 뒤에 터진다.

그래서 그 트랜잭션 바깥에 있는 호출자만 잡을 수 있고, 재시도할 때마다 새 트랜잭션으로 최신 버전을 다시 읽어야 한다.

4

분산락 — 상호 배제를 DB 밖으로 꺼낸다

Redis가 심판을 보고, 락을 얻은 요청만 DB로 간다

```
RLock lock = redissonClient.getLock("seat:lock:" + performanceId);
```

구조

락을 잡는 바깥 클래스와
@Transactional이 붙은 안쪽 클래스를
분리한다. 락 → 트랜잭션 → 커밋 →
락 해제 순서가 되어야 한다.

임대 시간

tryLock에 임대 시간을 주지 않았다.
Redisson 워치독이 작업 중에는
연장하고, 프로세스가 죽으면
연장을 멈춰 자동 해제된다.

결과

100건 중 10건 성공,
90건 매진, 좌석 10.
2단계와 같은 숫자.

만약 한 메서드 안에서 락을 잡고 풀면 — 스프링은 메서드가 끝날 때 커밋하므로 락이 커밋보다 먼저 풀린다.

다음 요청이 아직 반영되지 않은 좌석 수를 읽고, 락을 끝까지 쥐고 있었는데도 초과 예약이 난다. 3단계 재시도와 같은 이유다.

전체 결과

좌석 10석 · 앞의 다섯 행은 H2, 분산락 행은 Docker의 Postgres와 Redis

전략	요청	성공	매진	충돌	최종 카운터
락 없음	100	100	0	—	1
비관적 락	100	10	90	—	10
낙관적 락 (재시도 없음)	100	10	59	31	10
낙관적 락 (재시도 없음)	10	1	0	9	1
낙관적 락 + 재시도	10	10	0	0	10
분산락 (Redis)	100	10	90	—	10

붉게 표시한 두 행이 이 프로젝트의 요점이다.

첫 행은 버그 그 자체다. 네 번째 행에서는 요청이 수요를 넘지 않게 되자 그림이 뒤집힌다 — 한 건만 성공하고 아홉 자리가 비었는데도 테스트는 통과한다. 데이터는 정합하고 서비스는 쓸모없다. 재시도가 필요한 이유가 여기서만 보인다.

표가 가르지 못하는 것

비관적 락과 분산락은 같은 결과를 냈다 — 왜 그리고 언제 갈리는가

비관적 락

락을 DB가 잡는다.

순서를 기다리는 동안 커넥션을 쥔 채로
줄을 선다.

경합이 심해지면 커넥션 풀이 먼저 마른다.

분산락

DB를 건드리기 전에 락을 잡는다.

기다리는 동안에는 아무것도 쥐지 않고,
차례가 와야 커넥션을 가져간다.

같은 조건에서 풀이 마르지 않는다.

그래서 이번엔 측정하지 않았다

커넥션 풀을 인위적으로 줄이고 트랜잭션에 sleep을 넣으면 차이를 만들 수는 있다.

하지만 그건 관찰한 숫자가 아니라 테스트가 만들어낸 숫자다. 차이는 분명히 존재하고, 이 규모에서는 드러나지 않으며, 드러내려면 HTTP 계층을 통한 부하 테스트가 필요하다 — README에는 그렇게 적었다.

H2에서 Postgres로 옮기며 겪은 두 가지

둘 다 애플리케이션이 내놓은 에러만으로는 보이지 않았다

5432 포트를 두 프로세스가 함께 들고 있었다

예전에 Windows에 설치한 PostgreSQL 서비스가 도커의 포트 포워딩과 같은 주소에 묶여 있었다. Windows는 이 중복 바인딩을 막지 않는다. 먼저 응답한 쪽에는 ticket 계정이 없었고, 앱은 인증에 실패했다. 따로 확인한 것은 전부 정상이었다 — 컨테이너는 healthy, 설정은 일치, 컨테이너 안 psql은 접속 성공 (로컬 소켓은 비밀번호를 묻지 않는다). netstat이 리스너 두 개를 보여준 것이 유일한 단서였다.

Redisson 스타터가 Spring Boot 4와 맞지 않았다

redisson-spring-boot-starter는 Boot 3.5를 겨냥한 것이라, 4.1에서 RedisProperties를 찾지 못했다. 자동설정 클래스가 패키지를 옮겼기 때문이다. RedissonConfig에서 클라이언트를 직접 만들고 있었으므로 스타터가 기여하는 게 없었고, 스타터를 걷어내고 순수 라이브러리만 남겨 버전 결합 자체를 없앴다.

남는 문장 세 개

면접에서 이 프로젝트를 설명할 때 실제로 쓰이는 부분

1 트랜잭션은 격리를 주지 직렬화를 주지 않는다

@Transactional만으로는 lost update를 막을 수 없다. 읽기와 쓰기 사이의 틈이 문제의 뿌리다.

2 커밋보다 오래 살아야 하는 것은 트랜잭션 밖에 둔다

낙관적 락의 재시도도, 분산락의 락 해제도 같은 이유로 바깥 클래스에 있다.

3 정합성이 맞다고 서비스가 쓸 만한 것은 아니다

낙관적 락은 좌석이 남아 있는데도 거절했다. 테스트는 통과했고 서비스는 쓸모없었다.

남겨둔 것 — HTTP 계층과 k6 부하 테스트. 두 락 전략을 실제로 갈라놓을 부하 프로파일이 필요하다.